> Lab 1
Shell Script

TA: 葉弘堯

Prof.: 張軒彬

NCHU System & Network Lab

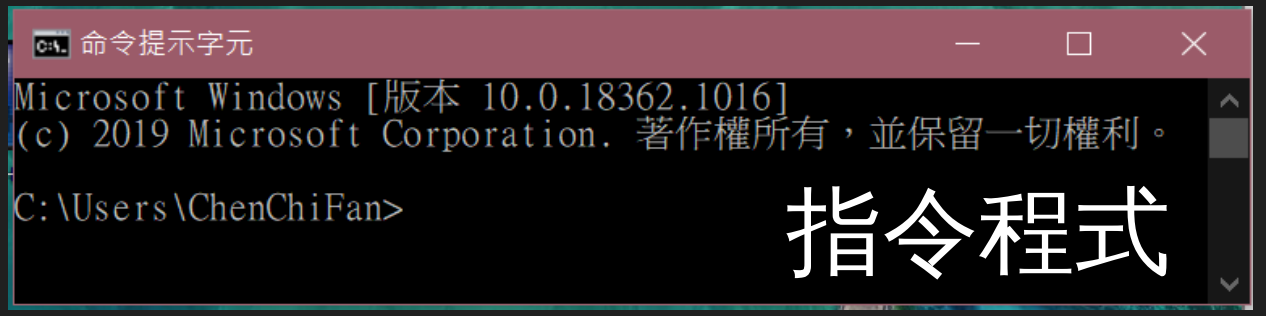
$ Outline

- Introduction
- Vim Basics
- Bash Shell Scripting Tutorial
- Exercise

$ Introduction (1/3)

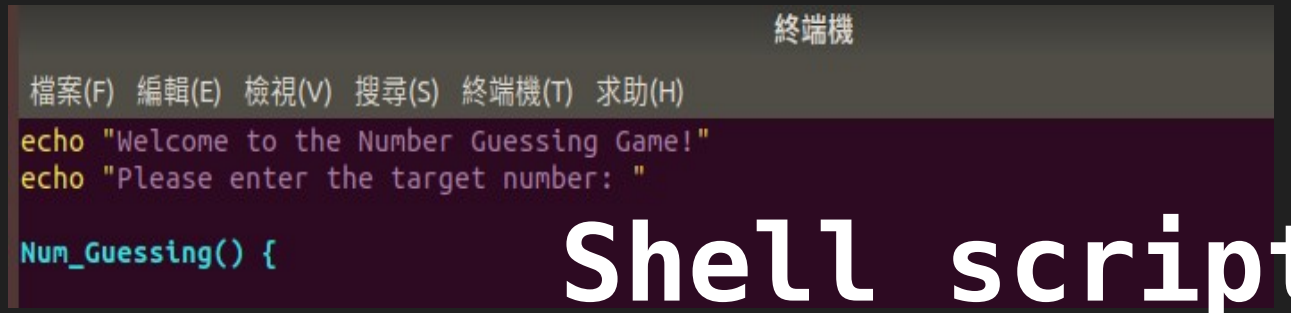
Window:

CMD



Linux:

shel



$ Introduction (2/3)

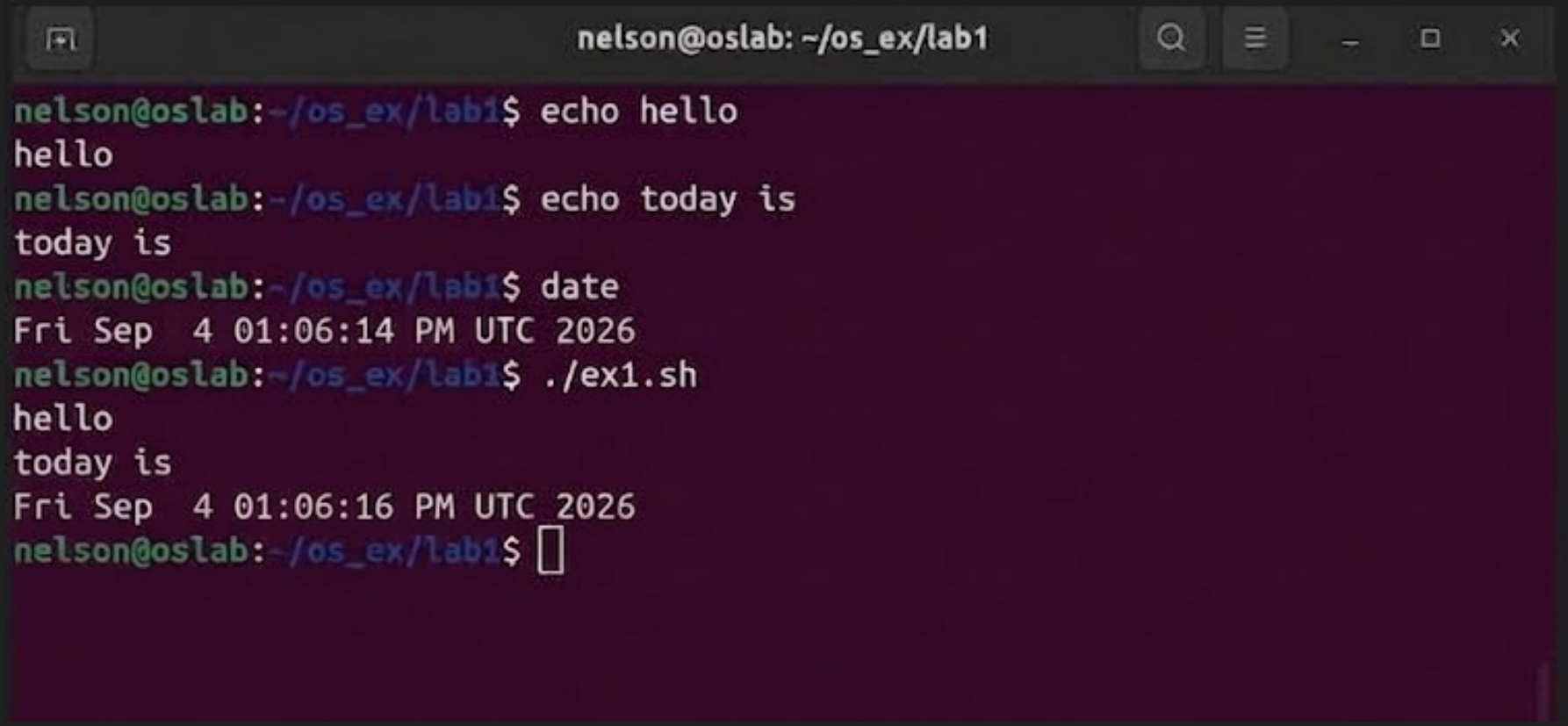
- shell script: 在 shell 裡我們通常只會一次下一個指令，shell 解讀這些指令後再讓 OS 幫我們執行。

- 我們也可以將多個指令寫在文字檔裡面，再透過 shell 解讀這個文字檔讓 OS 執行多個指令，這個文字檔就叫 shell script。

- 最常見的用處是將要重複執行的指令寫成一個檔案直接執行。

$ Introduction (3/3)

- 將三個指令包在一起執行



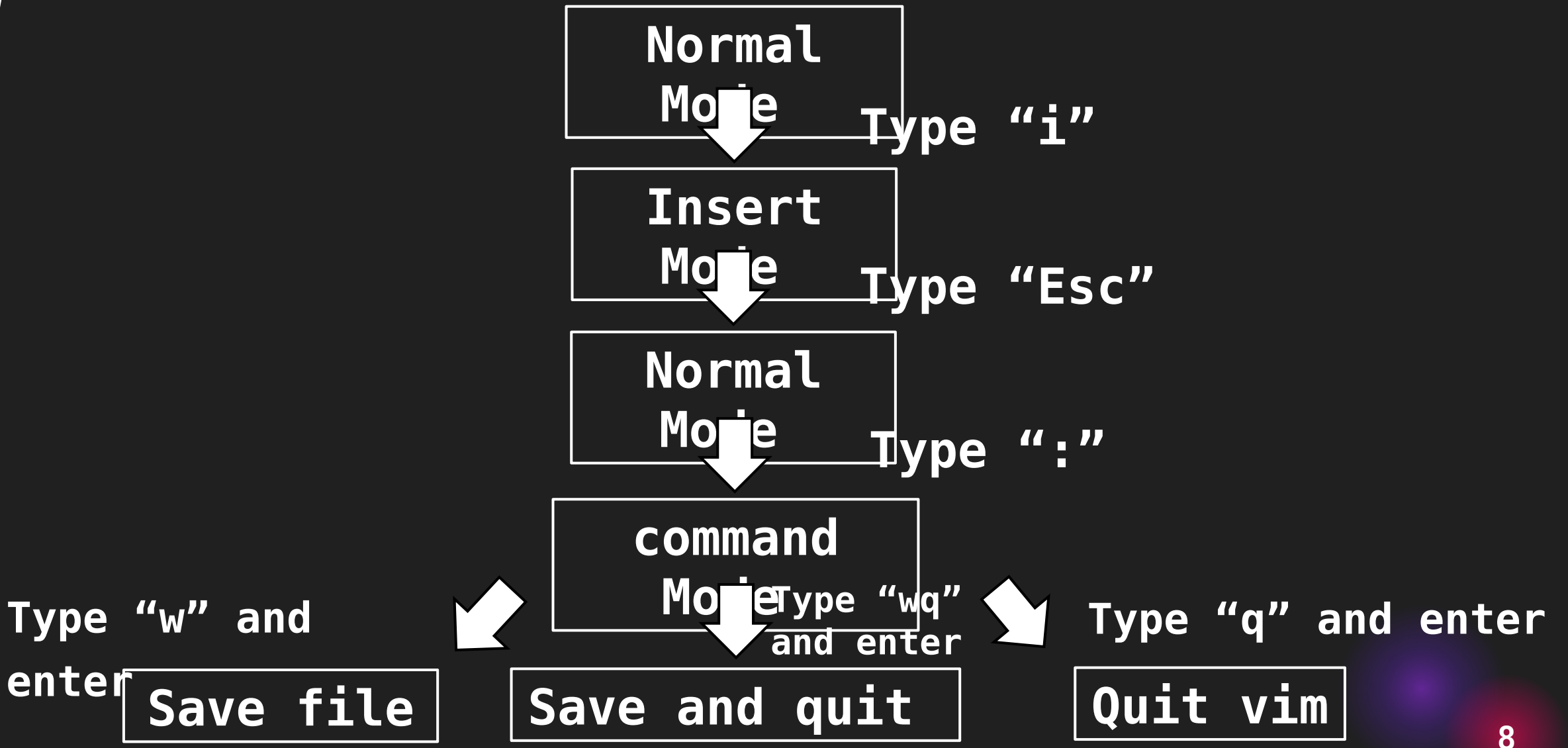
$ Outline

- Introduction
- Vim Basics
- Bash Shell Scripting Tutorial
- Exercise

$ Vim Basics (1/11)

- Vim 是一個文字編輯器
- 開啟 terminal: ctrl + alt + t
- 輸入 vim /路徑/檔名 （建立）開啟檔案（進入後在 normal mode）
- 按 i 進入編輯模式 (insert mode)
- 編輯完成後按 esc 進入一般模式 (normal mode)
- 一般模式下按 : 會進入命令模式 (command line mode)
- 命令模式下輸入 w 後按 enter 會儲存檔案 輸入 q 後按 enter 會離開 vim 輸入 wq 後按 enter 會儲存並退出

$ Vim Basics (2/11)



$ Vim Basics (3/11)

- 在 Shell 中安裝並更新 vim:
  - 輸入 **sudo apt-get update** 更新套件列表
  - 輸入 **sudo apt-get install vim** 安裝 vim 編輯器

$ Vim Basics (4/11)

- Hello vim !
  - Install Vim through **apt** command in shell
  - Create a file named “helloworld”
  - Type “Hello World!” using **insert mode**
  - Return to **normal mode** by pressing Ese
  - Save and quit the file

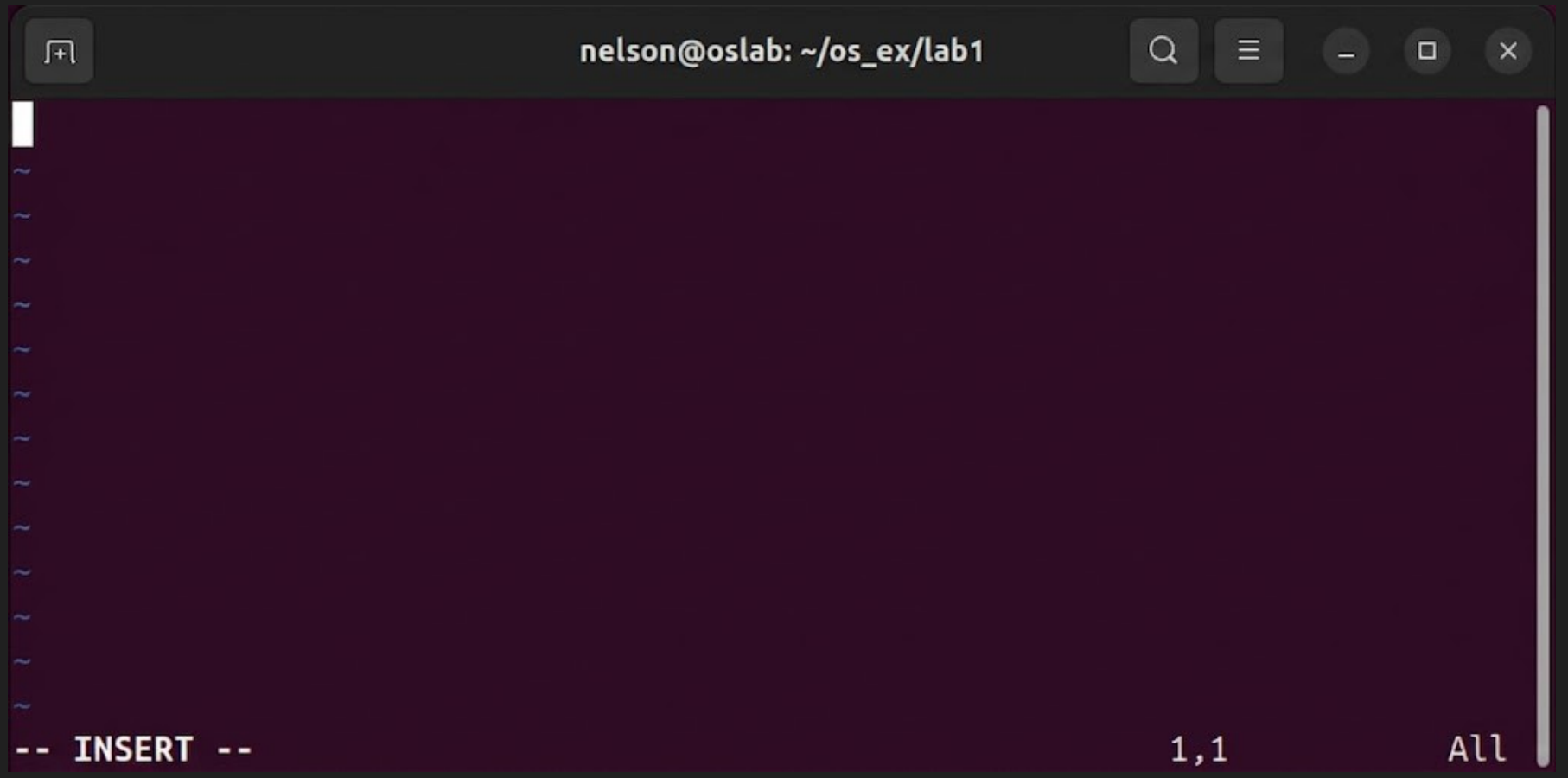
$ Vim Basics (5/11)

- Step1: Create a file named helloworld

```
nelson@oslab:~/os_ex/lab1$ vim helloworld
```

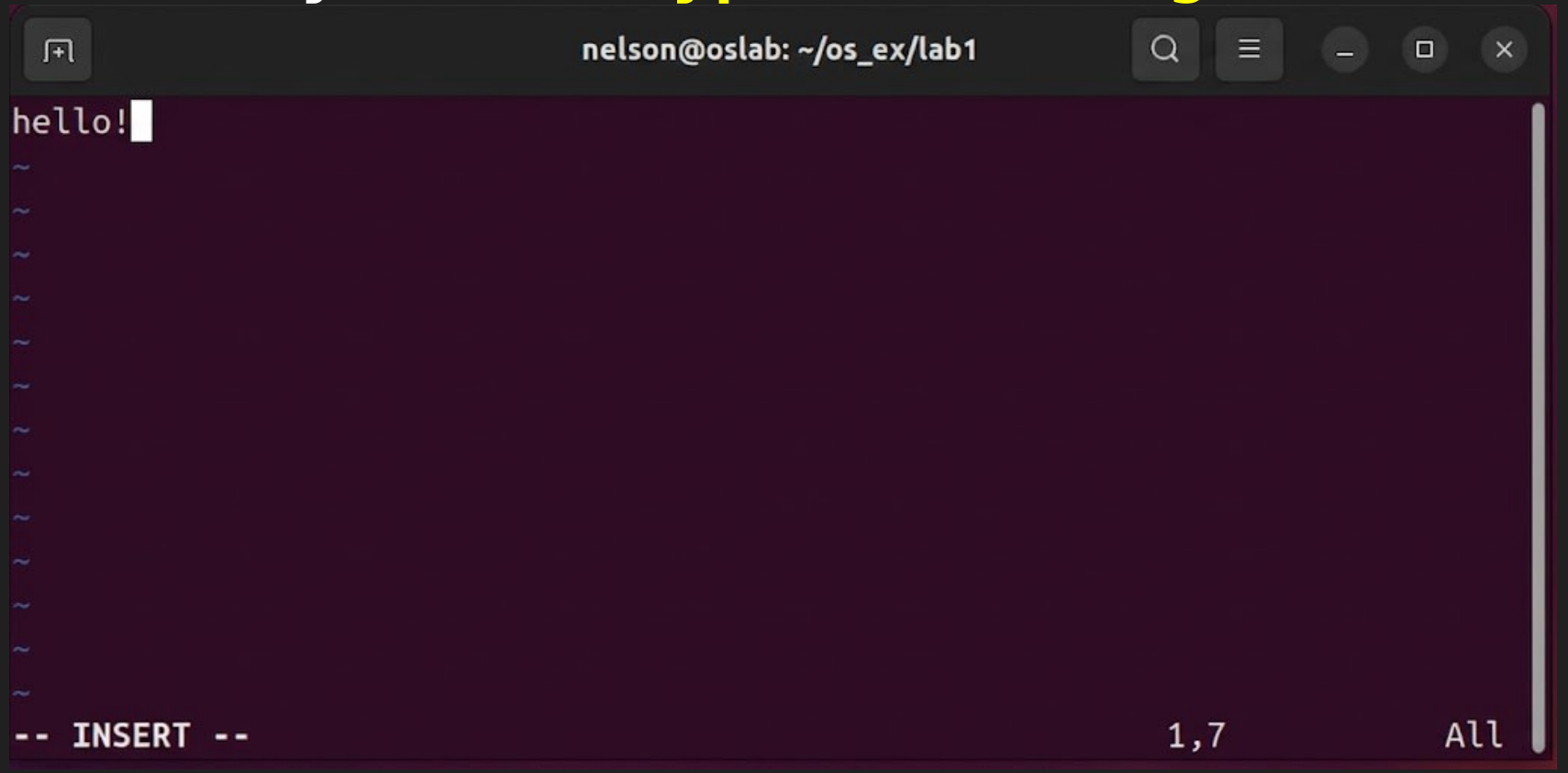
$ Vim Basics (6/11)

- Step2: **Enter Insert Mode** by pressing “i”



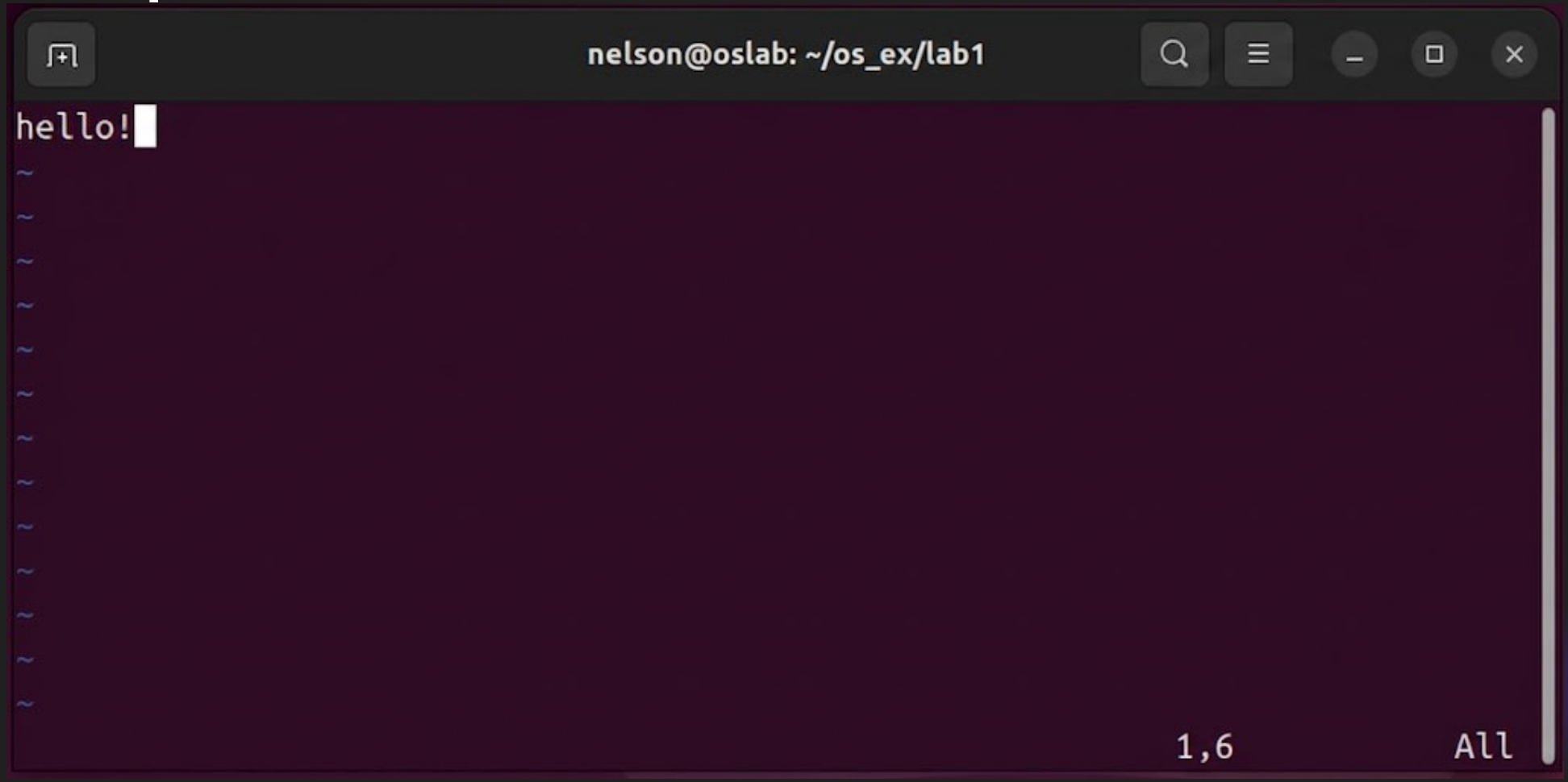
$ Vim Basics (7/11)

- Step3: Notice the --INSERT--” on bottom Left and you can type something now



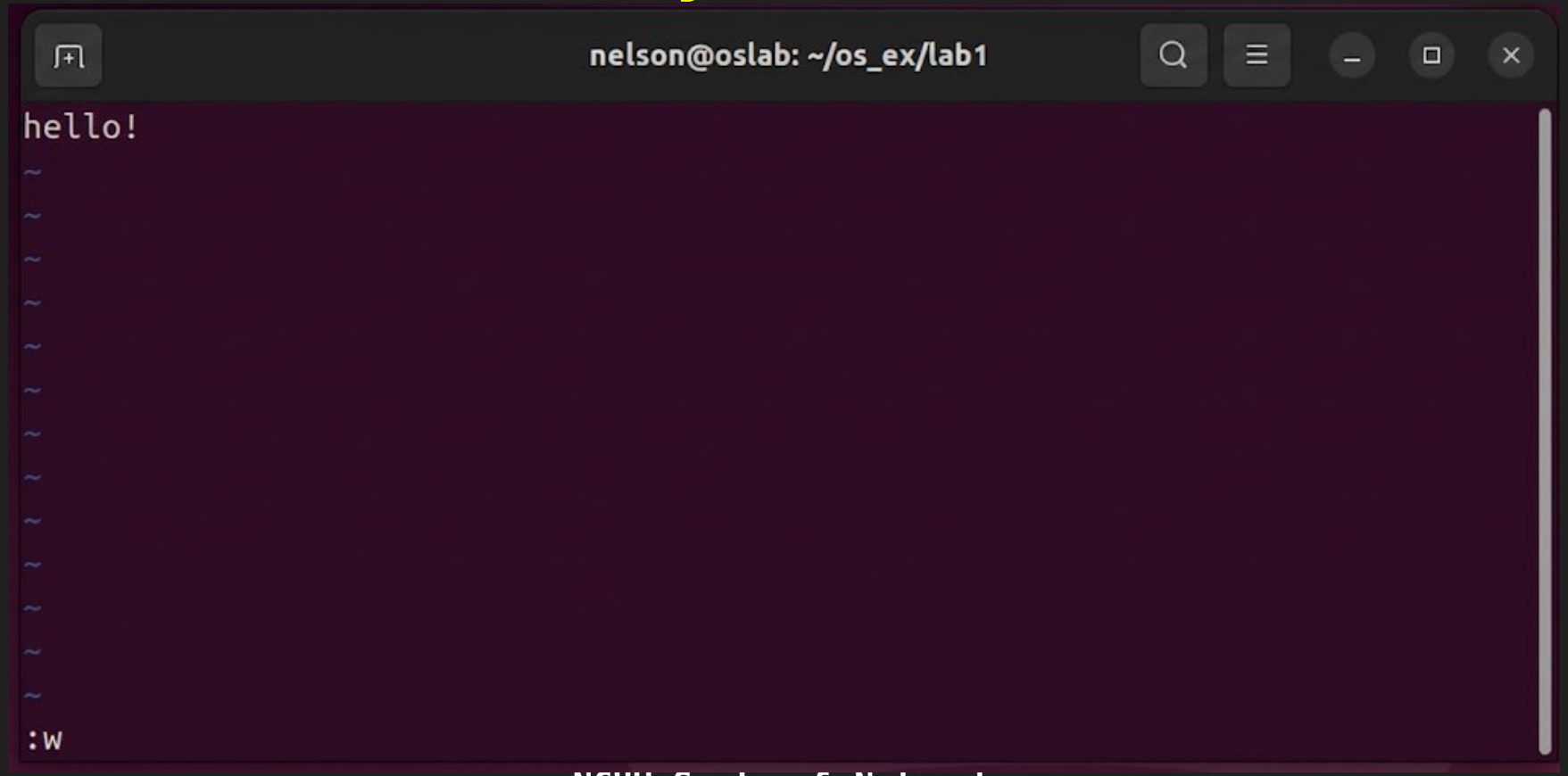
$ Vim Basics (8/11)

- Step4: Press “Esc” to return to normal mode



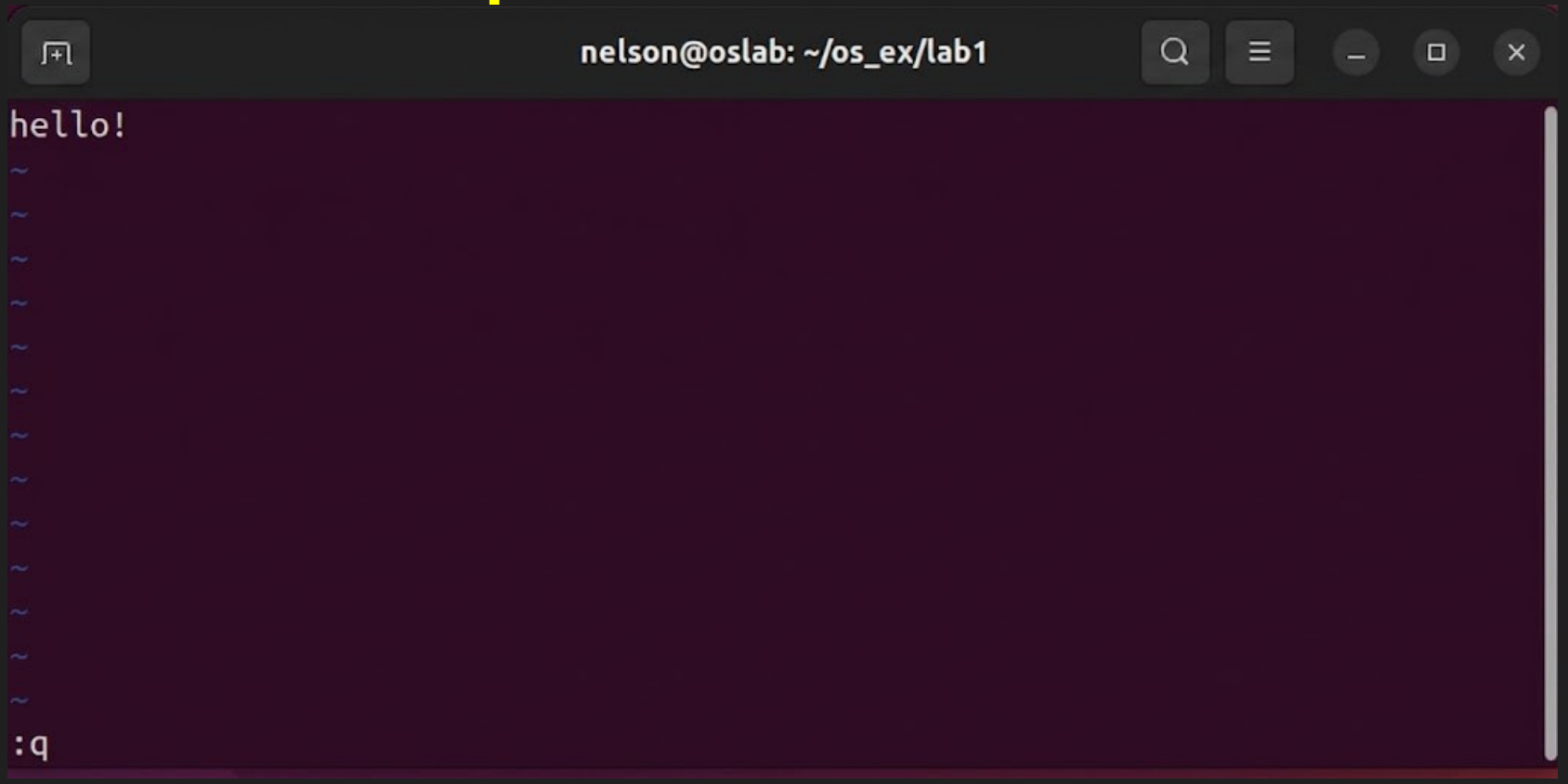
$ Vim Basics (9/11)

- Step5: Type “w” in and press “enter” command line mode to **save your file**



$ Vim Basics (10/11)

- Step6: Type “q” and press “enter” in command line mode to **quit Vim**



$ Vim Basics (11/11)

- Note
  1) Shell open : ctrl + alt + t
  2) vim open : vim helloworld.sh
  3) before typing : i
  4) Return to normal mode : Esc
  5) Save the File : w + Enter
  6) Quit Vim : q + Enter
  7) Save and Quit : wq + Enter

$ Outline

- Introduction
- Vim Basics
- **Bash Shell Scripting Tutorial**
- Exercise

$ Bash Shell Scripting Tutorial (1/28)

- **Bash :**
  - 為 **GNU** 專案所編寫的自由軟體 **Unix Shell**。
  - 為大多數 **Linux kernel** 系統、 **Mac OS X**、 **Darwin** 的默認 **Shell**。
  - 可以使用其他語言（**EX: Perl**、 **Python**）做到一樣的事情。

$ Bash Shell Scripting Tutorial (2/28)

- **echo**
- The simplest Hello World example
- Just simply type “echo Hello World!” in your shell and it’s done!

```
nelson@oslab:~/os_ex/lab1$ echo hello world!
hello world!
```

$ Bash Shell Scripting Tutorial (3/28)

- Hello World using .sh File
  - Now let’s use .sh file to do the same thing

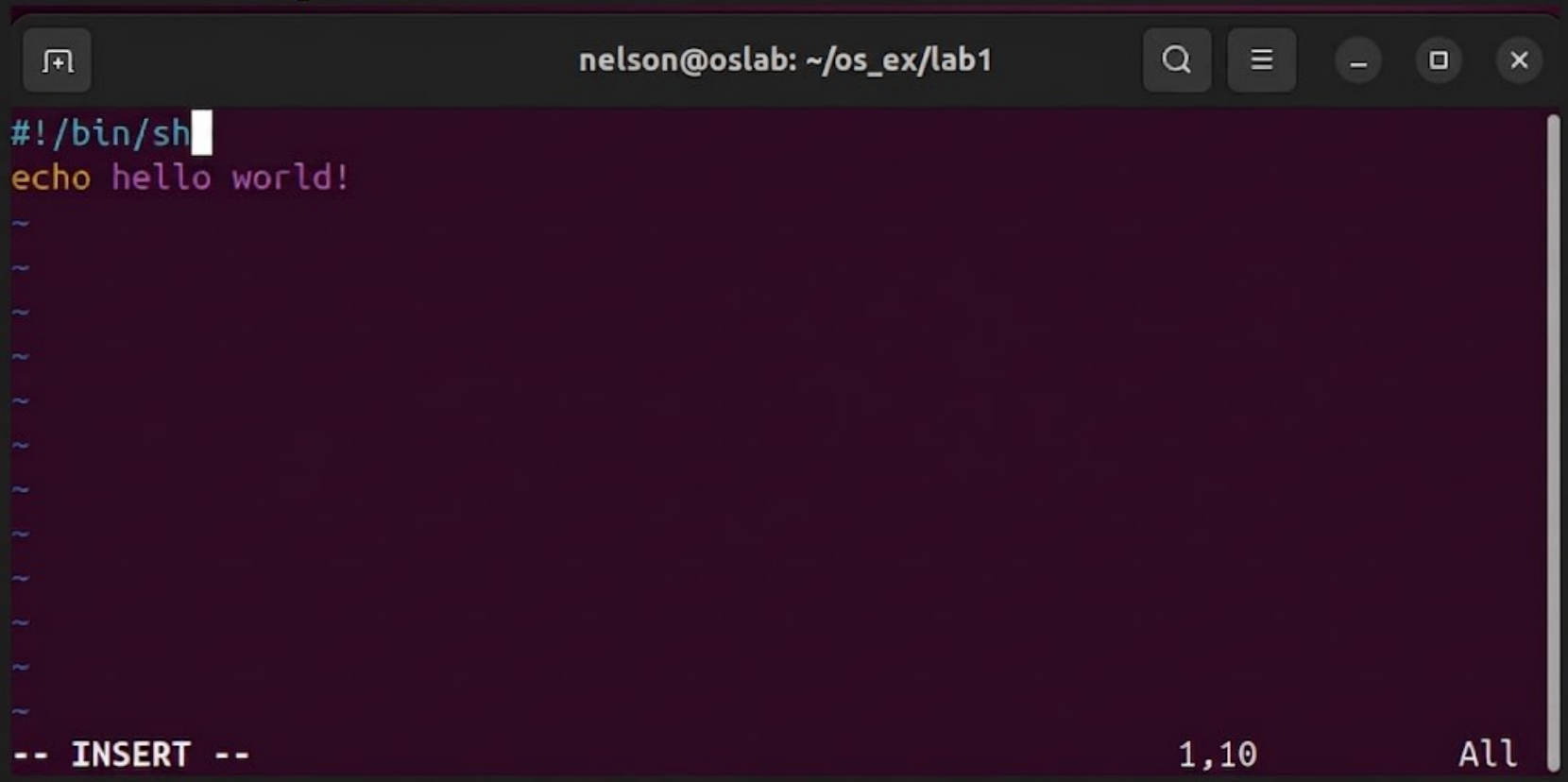
$ Bash Shell Scripting Tutorial (4/28)

- Step1: Type “vim helloworld.sh” in shell to **create a new shell file**

```
nelson@oslab:~/os_ex/lab1$ vim helloworld.sh
```

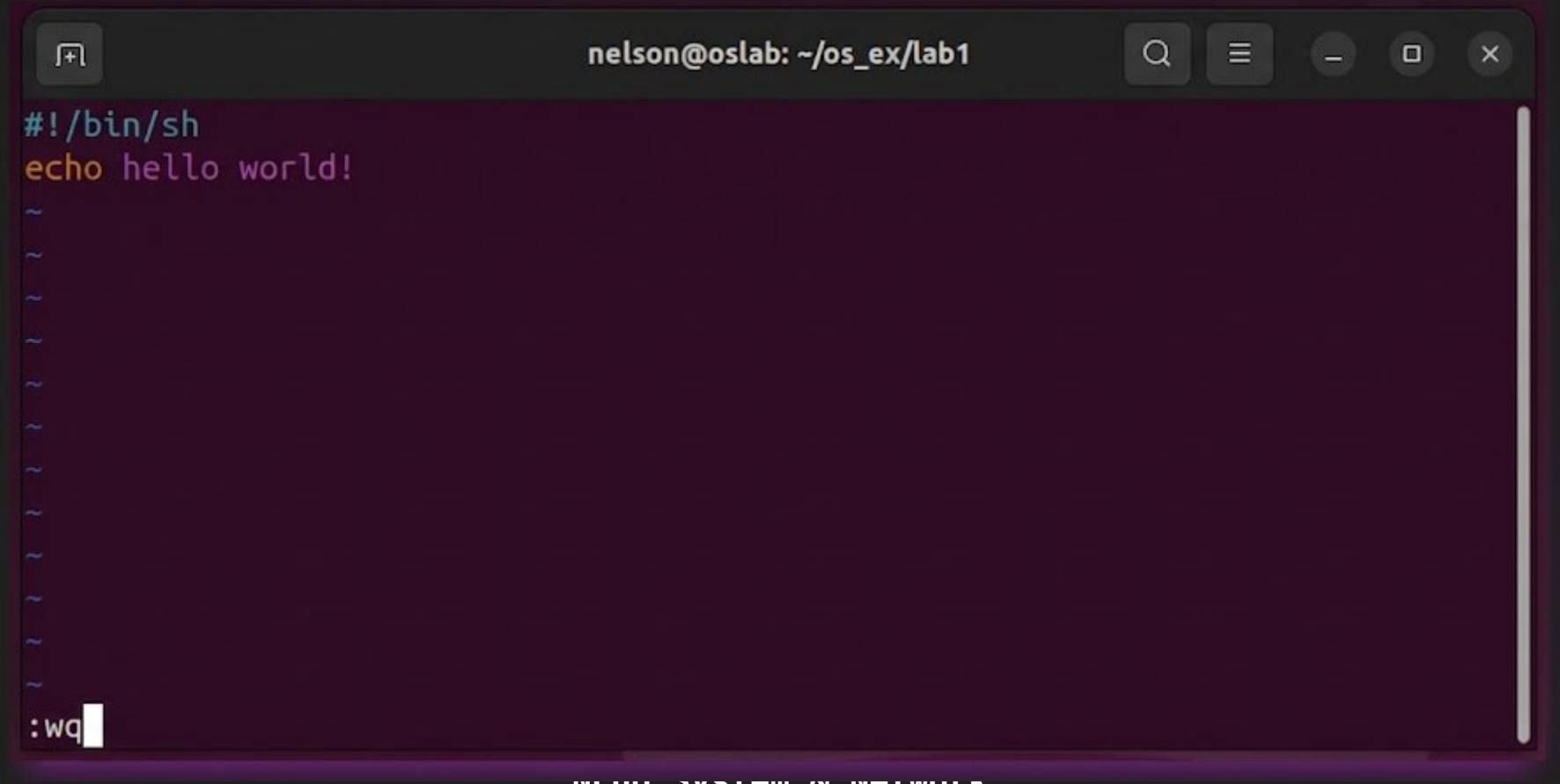
$ Bash Shell Scripting Tutorial (5/28)

- Step2: Enter the insert mode to **write our hello world script**



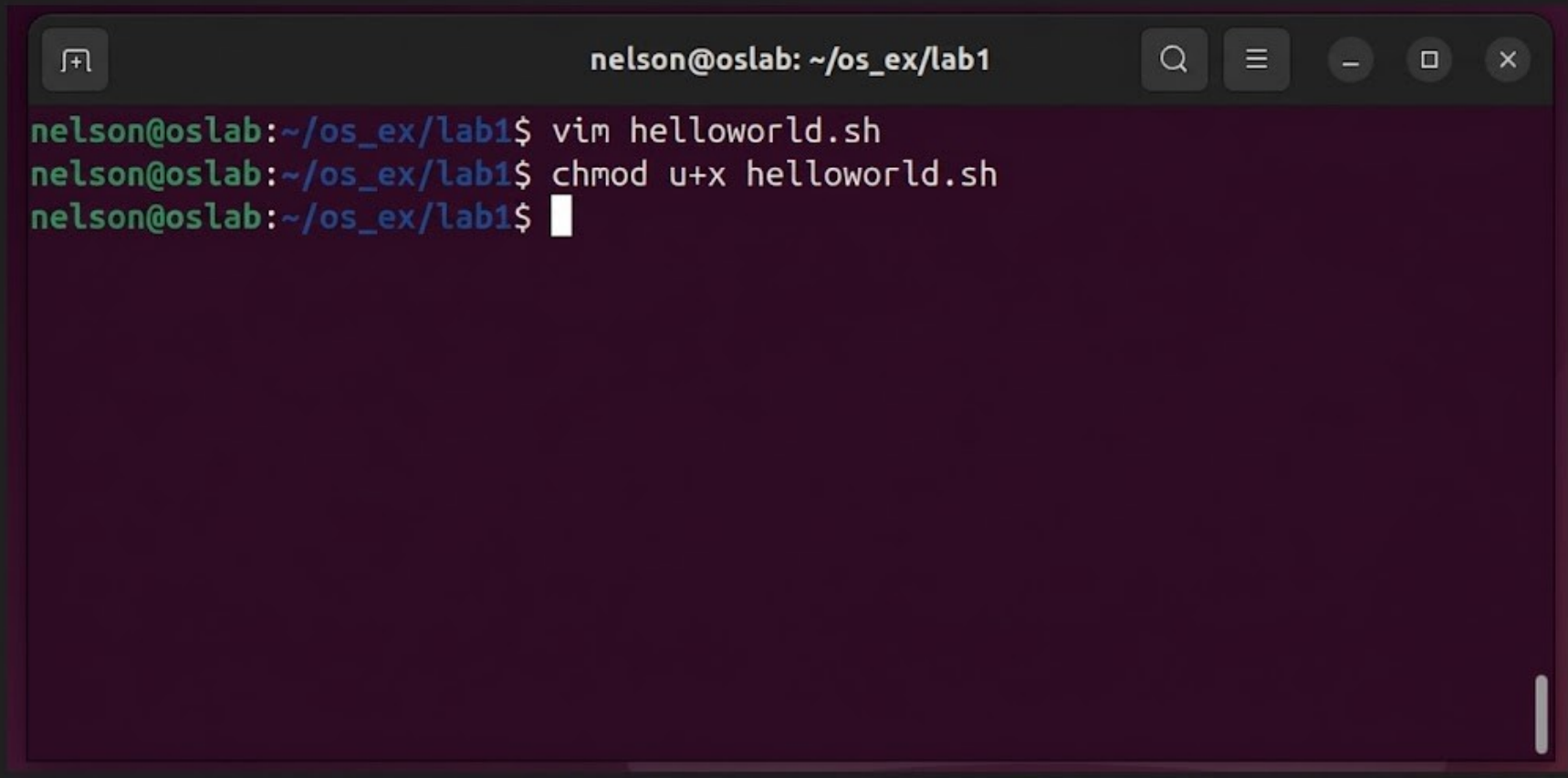
$ Bash Shell Scripting Tutorial (6/28)

- Step3: Return to the normal mode and type ":wq" to save and quit the file



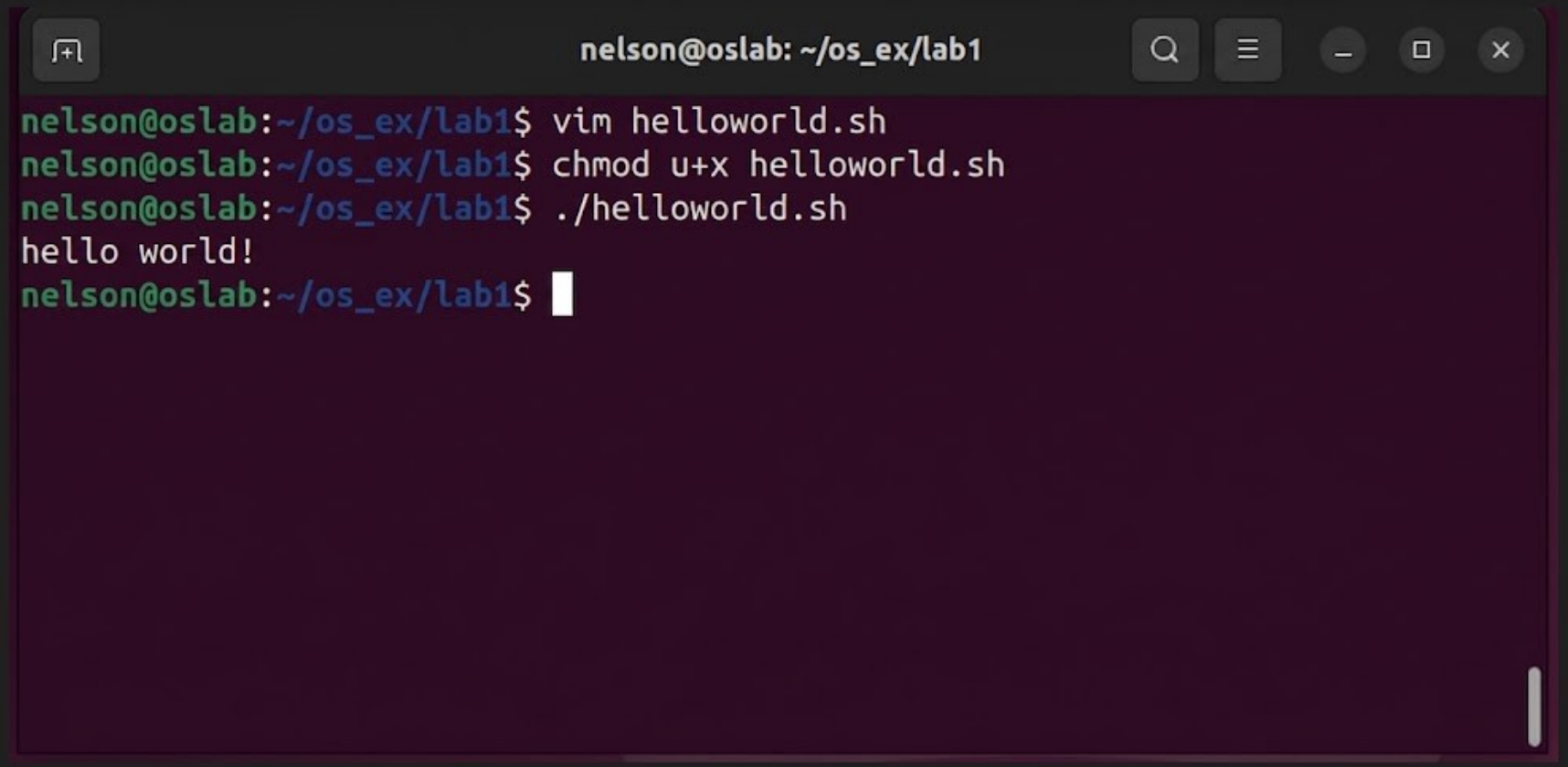
$ Bash Shell Scripting Tutorial (7/28)

- Step4: Use chmod command to **grant execute permission** to the shell file



$ Bash Shell Scripting Tutorial (8/28)

- Step5: Now we can **execute the script** directly by “./helloword.sh” command



$ Bash Shell Scripting Tutorial (9/28)

- Variable in Shell: There are two types of variable

1)System variables

```
nelson@oslab:~/os_ex/lab1$ echo $PWD
/home/nelson/os_ex/lab1
nelson@oslab:~/os_ex/lab1$ echo $SHELL
/bin/bash
```

- Ex: use “echo $PWD” to print your current working directory

```
nelson@oslab:~/os_ex/lab1$ no=10
nelson@oslab:~/os_ex/lab1$ echo $no
10
```

2)User defined variables (UDV)

- Ex: use “no=10” to set variable no and use “echo $no” to print 10 on your screen

$ Bash Shell Scripting Tutorial (10/28)

- Expr: shell arithmetic (+ - * /)
- Syntax: expr op1 math-operator op2

```
nelson@oslab:~/os_ex/lab1$ expr 1 + 3
4
nelson@oslab:~/os_ex/lab1$ expr 2 - 1
1
nelson@oslab:~/os_ex/lab1$ expr 10 \* 3
30
nelson@oslab:~/os_ex/lab1$ expr 12 / 4
3
```

space

在 shell 下 * 會被解讀成目前路徑下的所有檔名

加 \ 把他當一般字元解讀

$ Bash Shell Scripting Tutorial (11/28)

- More about Quotes
  - 雙引號("): 會解讀字串裡的變數（$） 與反引號（`），但會將多數特殊字元視為純文字。
  - 單引號('): 全部視為純文字。
  - 反引號(`): 執行引號內的指令並將結果放回原位

$ Bash Shell Scripting Tutorial (12/28)

- More about Quotes

```
nelson@oslab:~/os_ex/lab1$ var=hi
nelson@oslab:~/os_ex/lab1$ echo "$var, there"
hi, there
nelson@oslab:~/os_ex/lab1$ echo '$var, there'
$var, there
nelson@oslab:~/os_ex/lab1$ echo "expr 1 + 3"
expr 1 + 3
nelson@oslab:~/os_ex/lab1$ echo 'expr 1 + 3'
expr 1 + 3
nelson@oslab:~/os_ex/lab1$ echo "`expr 1 + 3`"
4
nelson@oslab:~/os_ex/lab1$ echo '`expr 1 + 3`'
`expr 1 + 3`
```

$ Bash Shell Scripting Tutorial (13/28)

- The read Statement
- Syntax: **read variable1 variable2 …**

```
nelson@oslab:~/os_ex/lab1$ read fname lname
nelson yeh
nelson@oslab:~/os_ex/lab1$ echo "hello,$fname $lname"
hello,nelson yeh
nelson@oslab:~/os_ex/lab1$
```

$ Bash Shell Scripting Tutorial (14/28)

- Arguments

- # myshell.sh

  echo “Welcome to use $0”

  echo “Hi, $1 $2, we have $# argument(s).”

- $ ./ex2.sh nelson yeh

  Welcome to use ./myshell.sh

  Hi, nelson yeh, we have 2 argument(s).

step1
step3
step4

```
nelson@oslab:~/os_ex/lab1$ vim ex2.sh
nelson@oslab:~/os_ex/lab1$ chmod u+x ex2.sh
nelson@oslab:~/os_ex/lab1$ ./ex2.sh nelson yeh
welcome to use ./ex2.sh
hi,nelson yeh,we have 2 argument(s)
```

step 2

willy@oslab: ~/os_ex/lab1

```
#!/bin/sh
echo "welcome to use $0"
echo "hi,$1 $2,we have $# argument(s)"
~
~
```

$ Bash Shell Scripting Tutorial (15/2

- **Conditionals**

```
nelson@oslab:~/os_ex/lab1$ vim ex3.sh
nelson@oslab:~/os_ex/lab1$ chmod u+x ex3.sh
nelson@oslab:~/os_ex/lab1$ ./ex3.sh
You can see me
```

- if-then
  - if [“foo” = “foo”]; then

        echo you can see me

    fi
  - if [“foo” = “bar”]; then

        echo you cannot see me

    fi

```
if [ "foo" = "foo" ]; then
        echo you can see me
fi

if [ "foo" = "bar" ]; then
        echo you cannot see me
fi
```

Linux shell中，
比較時，等號左右必須有空格
賦值時，等號旁邊不能有空格

$ Bash Shell Scripting Tutorial (16/28)

- test command or [expr]
  - used to see if an expression is true
  - returns 0 for true and nonzero for false
  - Use test command:

```
if test “foo” = “bar”;
then
    echo You cannot see me
fi
```

  - Use [expr]:

```
if [ “foo” = “bar” ];
then
    echo You cannot see me
fi
```

$ Bash Shell Scripting Tutorial (17/28)

- Conditionals
- Conditionals with variables

```
T1=”foo”
T2=”bar”
if [ “$T1” = “$T2” ]; then
        echo can see
else
        echo can not see
fi
```

```
T1="foo"
T2="bar"

if [ "$T1" = "$T2" ]; then
        echo can see
else
        echo can not see
fi
```

```
nelson@oslab:~$ vim ex3.sh
nelson@oslab:~$ ./ex3.sh
can not see
```

$ Bash Shell Scripting Tutorial (18/28)

- Conditionals
- if-then-elif-then-else

```
if [ “foo” = “bar” ];
then
    echo foo eq bar
elif [ “foo” = “foo” ];
  then
    echo foo eq foo
else
    echo foo not eq foo
fi
```

```
if [ "foo" = "bar" ]; then
        echo foo eq bar
elif [ "foo" = "foo" ]; then
        echo foo eq foo
else
        echo foo not eq foo
fi
```

```
nelson@oslab:~$ vim ex3.sh
nelson@oslab:~$ ./ex3.sh
foo eq foo
```

$ Bash Shell Scripting Tutorial (19/28)

- **Conditionals**
  - **Ex: if [“$a“ –lt “$b”]; then**

-eq	is equal to
-ne	is not equal to
-lt	is less than
-le	is less than or equal to
-gt	is greater than
-ge	is greater than or equal to

▲ Operators for Mathematics

$ Bash Shell Scripting Tutorial (20/28)

- **Conditionals**
  - **Ex: if [-n str1]; then**

str1 = str2	str1 is equal to str2
str1 != str2	str1 is NOT equal to str2
str1	str1 is NOT empty
-n str1	str1’s length>0
-z str1	str1’s length=0

▲ **Operators for Strings**

$ Bash Shell Scripting Tutorial (21/28)

- Loops
- for loop

- for loop
  - for i in 1 2 3 4 5; do
    echo Item $i
    done
  - for ((i=1; i<6; i++)); do
    echo Item $i
    done

```
for i in 1 2 3 4 5 ; do
        echo Item $i
done

for ((i=1 ; i<6 ; i++)); do
        echo Item $i
done
```

```
nelson@oslab:~$ vim ex4.sh
nelson@oslab:~$ chmod u+x ex4.sh
nelson@oslab:~$ ./ex4.sh
Item 1
Item 2
Item 3
Item 4
Item 5
Item 1
Item 2
Item 3
Item 4
Item 5
```

$ Bash Shell Scripting Tutorial (22/28)

- Loops
- while loop

```
i=1
while [ $i -lt 6 ];
  do
      echo Item $i
      i=`expr $i + 1`
done
```

```
i=1
while [ $i -lt 6 ]; do
        echo Item $i
        i=`expr $i + 1`
done
```

可用：

```
((i++))
((i=i+1))
((i+=1))
```

```
nelson@oslab:~$ vim ex4.sh
nelson@oslab:~$ ./ex4.sh
Item 1
Item 2
Item 3
Item 4
Item 5
```

$ Bash Shell Scripting Tutorial (23/28)

- The case Statement

- Syntax
  - case $var in
    pattern1)
    command;;
    pattern2)
    command;;
    patternN)
    command;;
    *) command;;

$ Bash Shell Scripting Tutorial (24/28)

- The case Statement

- Example: name.sh

```
case $1 in
        “Amy") echo “Amy Lin";;
        “Jimmy") echo “Jimmy Wang";;
        “Johnny") echo “Johnny Chen";;
        “Purple") echo “Purple Red";;
        *) echo “Error"
esac
```

```
echo "Enter a first name"
read first
case $first in
        "Amy")echo "Amy Lin";;
        "Jimmy")echo "Jimmy Wang";;
        "Johnny")echo "Johnny Chen";;
        "Purple")echo "Purple Red";;
        *)echo "Error"
esac
```

$ Bash Shell Scripting Tutorial (25/28)

- The case Statement

- Result

```
$ ./ex5.sh
Enter a first name
Amy
Amy Lin
```

```
nelson@oslab:~$ vim ex5.sh
nelson@oslab:~$ ./ex5.sh
enter a first name
Amy
Amy Lin
```

$ Bash Shell Scripting Tutorial (26/28)

- Function

- Example
  num=10
  PrintNumber() {
  echo “The number is $num!”
  }
  PrintNumber;
- Result
  The number is 10!

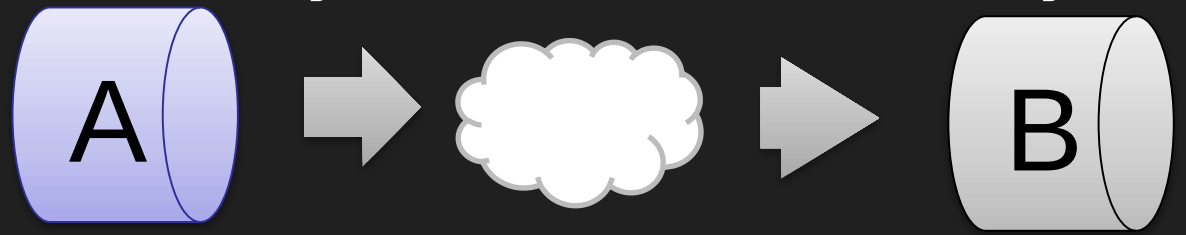
```
num=10

PrintNumber(){
        echo "The number is $num!"
}
PrintNumber;
```

```
nelson@oslab:~$ vim ex6.sh
nelson@oslab:~$ chmod u+x ex6.sh
nelson@oslab:~$ ./ex6.sh
The number is 10!
```

$ Bash Shell Scripting Tutorial (27/28)

- Pipes |
  - A pipe is a way to connect the output of one program to the input of another without any temporary file
  - Ex: We can use “ls | sort -r” to list sorted directory contents reversely




```
nelson@oslab:~/os_ex/lab1$ ls | sort -r
NGG.sh
helloworld.sh
helloworld
ex2.sh
ex1.sh
```

將 A 的輸出丟到 B 的輸入

$ Bash Shell Scripting Tutorial (28/28)

```
#!/bin/sh
#ex3.sh
echo "Amy"
echo "Jimmy"
echo "Johnny"
echo "Purple"
```

```
#!/bin/sh
#ex4.sh
while read first ; do
        case $first in
                "Amy")echo "Amy Lin";;
                "Jimmy")echo "Jimmy Wang";;
                "Johnny")echo "Jhonny Chen";;
                "Purple")echo "Purple Red";;
                *)echo "error"
        esac
done
```

```
nelson@oslab:~/os_ex/lab1$ ./ex3.sh | ./ex4.sh
Amy Lin
Jimmy Wang
Jhonny Chen
Purple Red
```

$ Outline

- Introduction
- Vim Basics
- Bash Shell Scripting Tutorial
- Exercise

$ Exercises (1/3)

- Number Guessing Game
  - Define the number range between 1 and 10
  - Ask the user to enter a target number at beginning
  - Keep narrowing down the number range until the player guessing the right target number

$ Exercises (2/3)

- Use **echo**, **read**, **loop** and **conditionals**
- Use **function** to ask for guessing number

$ Exercises (3/3)

- Result

```
willy@oslab:~/os_ex/lab1$ ./NGG.sh
Please enter the target number between 1 and 10:
5
Please guess a number between 1 and 10
8
Too high!
Please enter a number between 1 and 7
2
Too low
Please enter a number between 3 and 7
6
Too high!
Please enter a number between 3 and 5
3
Too low
Please enter a number between 4 and 5
5
Bingo!
```

$ References

- Shell script
- Advanced Packaging Tool (APT)
- New to Vim
- BASH Programming - Introduction
- Linux Shell Scripting Tutorial
- Common LINUX Command

$ 附錄

- 其他常用指令
  - 查看目錄 : ls
  - 目錄切換 : cd
  - 切換到 home 目錄 : cd ~
  - 查看當前目工作錄 : pwd
  - 創建目錄 : mkdir
  - 創建文件 : touch
  - 查看文件 : cat
  - 複製文件 / 文件夾 : cp
  - 移動文件 / 文件夾 : mv
  - 刪除文件 / 文件夾 : rm
  - 輸入指定內容 : echo
  - 更改權限 : chmod
  - 搜尋 : find
  - 檔案中找關鍵字 : grep

$ 附錄

- 其他常用快捷鍵
  - Ctrl + Shift + T : 開啟 terminal 分頁
  - Ctrl + L : 清空 terminal
  - Ctrl + C : 強制停止
  - Ctrl + D : 關閉
  - Ctrl + S : 鎖定
  - Ctrl + Q : 解除鎖定



Thanks for your listening